Task 3: Heuristic Graph Pathfinding Agent Search Engine

Progree Remote Internship: Artificial Intelligence
Intern: **Darakhshan Mujtaba**

1. Objective

The goal of this task was to build a search engine that lets a virtual agent find the shortest route through a grid maze. Two classic algorithms were implemented **from scratch** in Python: **Dijkstra's algorithm** and **A* search**. Both were tested on many mazes of different sizes and obstacle densities, measuring path length (steps), runtime and the number of expanded nodes, and the searches were exported as visualizations and a CSV log. All code was written and run in Google Colab.

2. Problem setup

- **Grid maze:** an N x N grid where each cell is free (0) or a wall (1). Walls are placed at random with a chosen probability (the obstacle ratio), using a fixed random seed so every experiment can be repeated exactly.
- **Agent:** starts in the top-left corner and must reach the bottom-right corner. It can move up, down, left or right, and every move costs 1 step.
- **Goal:** find the shortest possible path while checking (expanding) as few cells as possible.

3. Algorithms

3.1 Dijkstra's algorithm

Dijkstra's algorithm keeps a priority queue of cells ordered by their distance from the start, g(n). It always expands the closest unexplored cell first, so the search spreads out evenly in all directions like a wave. It is guaranteed to find the shortest path, but it has no sense of where the goal is, so it explores almost the whole maze.

3.2 A* search

A* ranks cells by f(n) = g(n) + h(n), where h(n) is a **heuristic**: an estimate of the remaining distance to the goal. With 4-directional movement the **Manhattan distance** |row difference| + |column difference| is used. It never overestimates the true distance (it is *admissible*), so A* is still guaranteed to find the shortest path, while it focuses the search towards the goal.

3.3 Implementation

A single function `search(grid, start, goal, use_heuristic)` implements both algorithms: with `use_heuristic=False` h(n) is 0 and the code behaves as Dijkstra; with True it is A*. It uses Python's `heapq` as the priority queue, a `came_from` dictionary to rebuild the path by walking backwards from the goal, and a list of expanded cells for the metrics and visualizations. No pathfinding library was used.

Tie-breaking optimisation. Many cells often share the same f-value. The first version broke these ties arbitrarily. The improved version stores h(n) as a second priority, so among equally good cells the one closer to the goal is expanded first. This does not change the path length, but greatly reduces wasted expansions (Section 5.3).

4. Experimental design

Parameter	Values
Grid sizes	20 x 20, 40 x 40, 60 x 60
Obstacle ratios	10%, 20%, 30% walls
Mazes per setting	5 solvable mazes (mazes with no possible path were skipped)
Total	45 mazes, each solved by both algorithms (90 runs)
Metrics	Path steps, nodes expanded, expanded-node density (% of free cells checked), runtime in ms

Every run was written to an exported log file, `pathfinding_log.csv`, with one row per run.

5. Results

5.1 Summary of all experiments

Grid	Walls	Steps	Expanded (Dijkstra)	Expanded (A*)	Density D / A* (%)	Runtime D / A* (ms)
20x20	10%	38	362.2	**61.4**	99.9 / **16.9**	1.57 / **0.34**
20x20	20%	38	322.8	**59.6**	99.6 / **18.4**	1.10 / **0.25**
20x20	30%	40	237.6	**94.6**	84.8 / **34.2**	0.74 / **0.36**
40x40	10%	78	1,441.4	**120.6**	100.0 / **8.4**	5.62 / **0.57**
40x40	20%	78	1,269.0	**143.2**	99.4 / **11.2**	4.31 / **0.62**
40x40	30%	79.6	1,102.4	**230.4**	97.6 / **20.5**	3.57 / **0.93**
60x60	10%	118	3,224.0	**294.4**	100.0 / **9.1**	12.75 / **1.45**
60x60	20%	118	2,846.4	**285.8**	99.7 / **10.0**	11.24 / **1.27**
60x60	30%	121.2	2,432.0	**666.0**	96.4 / **26.4**	8.70 / **2.84**

Table 2. Averages over 5 mazes per setting. D = Dijkstra. Runtimes measured in Google Colab.

In every one of the 45 mazes both algorithms found a path of exactly the same length, which confirms that A* with the Manhattan heuristic is optimal. Dijkstra checked 85 to 100% of the free cells, while A* checked only 8 to 34%. Across all experiments A* expanded **85% fewer nodes** than Dijkstra, and on the largest 60 x 60 mazes it was on average **5.9 times faster**.

5.2 Visualization of the search

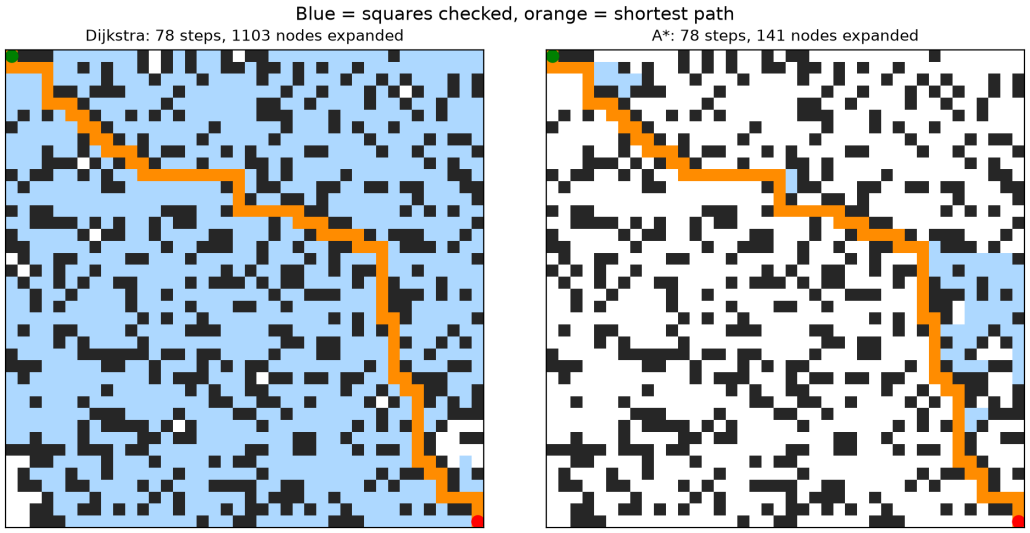


Figure 1. The same 40 x 40 maze (30% walls) solved by both algorithms. Blue = expanded cells, orange = shortest path, green = start, red = goal.

Figure 1 shows the difference clearly. Dijkstra floods almost the entire maze before reaching the goal (1,103 expanded cells), while A* moves almost directly along the optimal route (141 expanded cells). Both paths are 78 steps long.

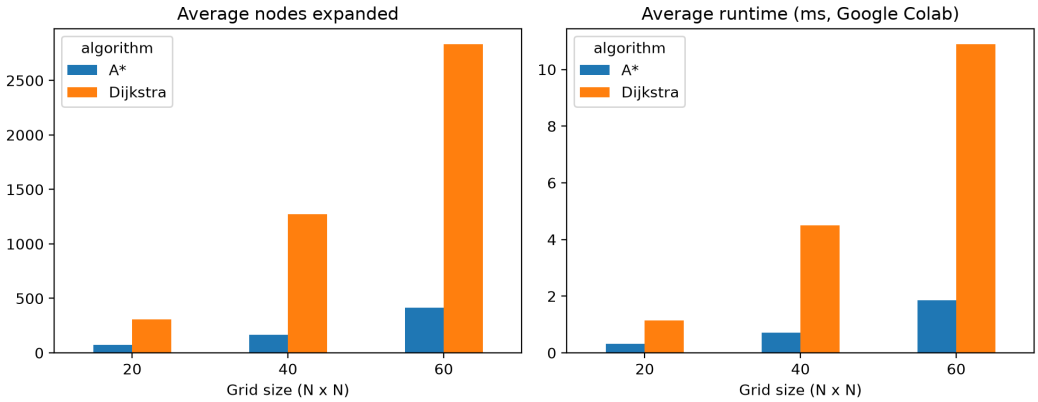


Figure 2. Average nodes expanded and runtime for each grid size.

As the grid grows, Dijkstra's work grows with the area of the maze (about N squared), while A*'s work grows much more slowly, roughly with the length of the path. The gap therefore widens on bigger mazes.

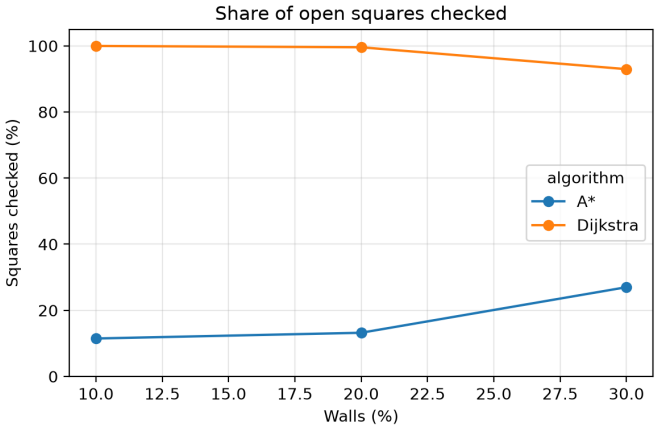


Figure 3. Share of free cells checked, by obstacle ratio.

More walls make A* work harder: obstacles block the straight line to the goal, so the heuristic becomes less accurate and A* has to explore detours. Dijkstra's share falls slightly at 30% walls only because some free cells are cut off from the start.

5.3 Effect of tie-breaking

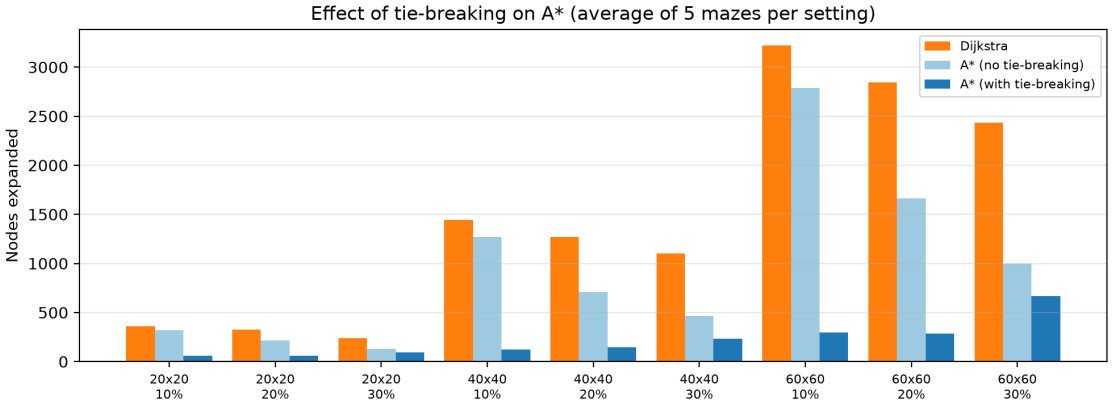


Figure 4. Nodes expanded by Dijkstra, A without tie-breaking, and A* with tie-breaking.*

Without tie-breaking, A* still expanded 39 to 89% of the free cells, because many cells had equal f-values and were explored in no useful order. Adding h(n) as a tie-breaker reduced A*'s expansions by **77%** overall with no change in path length. This shows that small implementation details can matter as much as the choice of algorithm.

6. Conclusion

I implemented Dijkstra's algorithm and A* search from scratch and evaluated them on 45 grid mazes of different sizes and obstacle densities. Both always find the optimal path, but A* with a Manhattan-distance heuristic and tie-breaking explores far fewer cells and runs several times faster, and its advantage grows with the size of the maze. The exported visualizations and CSV log make the optimisation directly visible.

Possible extensions:

- Allow diagonal moves and use the Octile or Euclidean distance as the heuristic.
- Compare with a Q-Learning agent, which learns a route by trial and error instead of searching.
- Add weighted terrain (cells that cost more to cross) and moving obstacles.

Appendix: Source code (run in Google Colab)

```
# Task 3: Heuristic Grid Pathfinding (Dijkstra and A*), code as run in Google Colab

# ---- Step 1: build a grid maze
import numpy as np
import matplotlib.pyplot as plt

def make_grid(size=30, obstacle_ratio=0.25, seed=0):
    rng = np.random.default_rng(seed)
    grid = (rng.random((size, size)) < obstacle_ratio).astype(int)  # 1 = wall, 0 = free
    grid[0, 0] = grid[size - 1, size - 1] = 0                        # keep start and goal free
    return grid

def show(grid, title="Grid maze"):
    plt.figure(figsize=(5, 5))
    plt.imshow(grid, cmap="Greys")
    plt.scatter([0], [0], c="green", s=80, label="Start")
    plt.scatter([grid.shape[1] - 1], [grid.shape[0] - 1], c="red", s=80, label="Goal")
    plt.title(title); plt.legend(loc="upper right"); plt.xticks([]); plt.yticks([])
    plt.show()

grid = make_grid(30, 0.25)
print("Grid size:", grid.shape, "| walls:", grid.sum(), "| free cells:", (grid == 0).sum())
show(grid, "30x30 maze, 25% walls")

# ---- Step 2: Dijkstra and A* from scratch (with tie-breaking)
import heapq, time

def manhattan(a, b):
    return abs(a[0] - b[0]) + abs(a[1] - b[1])

def search(grid, start, goal, use_heuristic=True):
    """A* when use_heuristic=True, Dijkstra when False. Moves: up, down, left, right."""
    t0 = time.perf_counter()
    rows, cols = grid.shape
    open_list = [(0, 0, start)]           # priority queue of (priority, tie-breaker, cell)
    came_from = {start: None}             # to rebuild the path at the end
    cost = {start: 0}                     # steps from start to each cell
    expanded = []                         # cells we fully explored (in order)
    done = set()
    while open_list:
        _, _, cell = heapq.heappop(open_list)
        if cell in done:
            continue
        done.add(cell)
        expanded.append(cell)
        if cell == goal:
            break
        r, c = cell
        for nr, nc in [(r - 1, c), (r + 1, c), (r, c - 1), (r, c + 1)]:
            if 0 <= nr < rows and 0 <= nc < cols and grid[nr, nc] == 0:
                new_cost = cost[cell] + 1
                if new_cost < cost.get((nr, nc), float("inf")):
                    cost[(nr, nc)] = new_cost
                    came_from[(nr, nc)] = cell
                    h = manhattan((nr, nc), goal) if use_heuristic else 0
                    heapq.heappush(open_list, (new_cost + h, h, (nr, nc)))
    path = []
    if goal in came_from:                 # walk backwards from goal to start
        node = goal
        while node is not None:
            path.append(node)
            node = came_from[node]
        path.reverse()
    runtime_ms = (time.perf_counter() - t0) * 1000
    return path, expanded, runtime_ms
start, goal = (0, 0), (grid.shape[0] - 1, grid.shape[1] - 1)
for name, h in [("Dijkstra", False), ("A*", True)]:
    path, expanded, ms = search(grid, start, goal, use_heuristic=h)
    print(f"{name:9s} path steps={len(path) - 1:3d} | nodes expanded={len(expanded):4d} | runtime={ms:.2f} ms")

# ---- Step 3: test on 45 mazes and export a log
import pandas as pd

results = []
for size in [20, 40, 60]:
    for ratio in [0.10, 0.20, 0.30]:
        seed, kept = 0, 0
        while kept < 5:                               # 5 solvable mazes per setting
            g = make_grid(size, ratio, seed)
            s, t = (0, 0), (size - 1, size - 1)
            seed += 1
            if not search(g, s, t)[0]:                # no path exists: skip this maze
                continue
            kept += 1
            for name, h in [("Dijkstra", False), ("A*", True)]:
```

```
                path, expanded, ms = search(g, s, t, use_heuristic=h)
                results.append({"size": size, "obstacles_%": int(ratio * 100), "seed": seed - 1, "algorithm": name,
                                "steps": len(path) - 1, "expanded": len(expanded),
                                "density_%": round(100 * len(expanded) / (g == 0).sum(), 1),
                                "runtime_ms": round(ms, 3)})

df = pd.DataFrame(results)
df.to_csv("pathfinding_log.csv", index=False)        # exported log of every run
summary = df.groupby(["size", "obstacles_%", "algorithm"])[["steps", "expanded", "density_%", "runtime_ms"]].mean().
      round(2)
print(summary)

# ---- Step 4: visualizations
# Picture 1: what each algorithm explored, and the path it found
def draw_search(ax, g, path, expanded, title):
    img = np.ones((*g.shape, 3))                # white = open
    img[g == 1] = [0.15, 0.15, 0.15]            # dark = wall
    for r, c in expanded:
        img[r, c] = [0.68, 0.85, 1.0]           # light blue = checked
    for r, c in path:
        img[r, c] = [1.0, 0.55, 0.0]            # orange = final path
    ax.imshow(img)
    ax.scatter([0], [0], c="green", s=60); ax.scatter([g.shape[1] - 1], [g.shape[0] - 1], c="red", s=60)
    ax.set_title(title, fontsize=10); ax.set_xticks([]); ax.set_yticks([])

s, t = (0, 0), (39, 39)
seed = 0
while not search(make_grid(40, 0.30, seed), s, t)[0]:   # pick the first maze that has a path
    seed += 1
g = make_grid(40, 0.30, seed)
fig, axes = plt.subplots(1, 2, figsize=(10, 5))
for ax, (name, h) in zip(axes, [("Dijkstra", False), ("A*", True)]):
    path, expanded, ms = search(g, s, t, use_heuristic=h)
    draw_search(ax, g, path, expanded, f"{name}: {len(path)-1} steps, {len(expanded)} nodes expanded")
plt.suptitle("Blue = squares checked, orange = shortest path")
plt.tight_layout(); plt.savefig("path_comparison.png", dpi=200); plt.show()

# Picture 2: average nodes expanded and runtime for each maze size
avg = df.groupby(["size", "algorithm"])[["expanded", "runtime_ms"]].mean().unstack()
fig, axes = plt.subplots(1, 2, figsize=(10, 4))
avg["expanded"].plot(kind="bar", ax=axes[0], rot=0, color=["#1f77b4", "#ff7f0e"])
axes[0].set_title("Average nodes expanded"); axes[0].set_xlabel("Grid size (N x N)")
avg["runtime_ms"].plot(kind="bar", ax=axes[1], rot=0, color=["#1f77b4", "#ff7f0e"])
axes[1].set_title("Average runtime (ms)"); axes[1].set_xlabel("Grid size (N x N)")
plt.tight_layout(); plt.savefig("metrics_by_size.png", dpi=200); plt.show()

# Picture 3: how wall density changes the search
dens = df.groupby(["obstacles_%", "algorithm"])["density_%"].mean().unstack()
dens.plot(marker="o", figsize=(6, 4), color=["#1f77b4", "#ff7f0e"])
plt.title("Share of open squares checked"); plt.xlabel("Walls (%)"); plt.ylabel("Squares checked (%)")
plt.ylim(0, 105); plt.grid(alpha=0.3)
plt.tight_layout(); plt.savefig("density_by_obstacles.png", dpi=200); plt.show()
```